

Okay, hello everyone.

Welcome back to Deep Learning Systems, Algorithms and Implementation.

I'm Tianqi Chen.

Today I'm going to work with everybody to talk about our lectures on neural network library abstractions.

So, up until now we have talk about a few of the basic deep learning models like multilayer perceptrons as well as the basic automatic differentiation algorithms.

Hopefully now you have started to work on your homework one and are having fun building some of these automatic differentiation libraries.

Today we're going to talk about how we can compose the things we build to really form an end to end neural network, or end to end deep learning library.

Up until now when we talk about the needle library, the only thing we've built so far, it only contains basic elements like tensors and automatic differentiation.

And there are a lot of things we can build on top so that we can build different kind of machine learning model more easily.

And today's lecture is going to contain two parts.

In the first part I'm going to quickly describe and discuss with folks on possible programming abstraction

we can use to build machine learning models, and actually do a bit of history review of the history of deep learning frameworks.

And the second part, I'm going to work with everybody on study some of the high level modular components

and how we can effectively decompose machine learning library design onto different architectural components that can then organically work together.

Okay?

Let's start with the first part.

So, what is programming abstraction?

So, the program abstraction involves things like API we use to define computations, how we can extend the overall framework to support things like GPUs and different kinds of models.

If you have used libraries like PyTorch or TensorFlow or another library before, some of those design choices may seem very obvious for you after you have seen and used them for a while.

But you know, if we try to, let's try to, you know, time rewind, and try to travel back to the time where those deep learning libraries are not yet available.

And you can find that some of our choices may not become as obvious, and it's very interesting to see how the deep learning framework abstractions evolves over time.

It's also important to recap and learn through this thought process on how we eventually arrive at the interfaces we are at today

because that helps us to know why the abstractions are design this way, and also, you know, as the field of machine learning, and also machine learning framework evolves, there's always a need of, you know, designing new capabilities and as a result there are, you know, new lessons.

These lessons can be used to help us to design new abstractions in the future as well.

Okay?

And so, in the first part we will do a case study of effectively, the history of the deep learning framework development,

and we are going to specifically focus on three past frameworks that are representative in the history of deep learning framework development.

So, the first framework we're going to look at is Caffe 1.0.

So, this is one of the earlier popular frameworks that help us to do most of the computer vision tasks on GPUs.

And you know, even today it's still being used in quite a few use cases.

Of course, most of the use cases have been switched to the later generation frameworks.

The second framework we're going to look at is TensorFlow 1.0.

And this is specifically interesting when we look at the particular design choices that comes into the TensorFlow 1.0 design.

And finally, we're going to look at PyTorch, or you know, today when we're looking at the library like needle, the programming abstraction where building really mirrors what PyTorch is doing.

So, these three frameworks were, you know, quite popular and dominated the market for a period of a time in the history of machine learning frameworks.

Of course there are many frameworks being developed along the way besides this three frameworks, right?

So, for example there's earlier pioneer the the field Theano, and Torch7 in Lua, and there's also a framework I was involved in called MxNet.

Caffe had a second generation framework called Caffe 2, Chainer, and more lately there's a very exciting development from Google that leads to Jax.

So, due to time we will not study all the frameworks, but a lot of them also share some of the similar traits that we will mention as we do the case study.

And that if you are more interested, you should go ahead and check how those frameworks as well and think about you know, what are the design choices in those frameworks specifically?

How do you design the interface to build neural networks?

And how are they different from each other?

And why they come come down with this kind of design?

Okay, so as a first step, let's first take a look at Caffe 1.0, and it's a typical representation of a first generation deep learning framework.

So in 2014, I think 12, yeah, 2012, the image Alex Krizhevsky and Geoffrey Hinton, and Ilya won the first, using the neural nets to win ImageNet challenge, right?

So, that's the inception of the deep learning wave in computer vision, and later on, all different kind of applications.

And one of the earlier pioneer in there is this framework called Cuda-Convnet that Alex Krizhevsky actually built to allow us to go ahead and compose neural networks, and use that for end to end training.

And Caffe is a system developed in Berkeley that tries to effectively formalize some of the engineering approaches, and then go ahead and you know, allow us to build different kind of machine learning models.

They are becoming very popular and almost, I think 80% of the deep learning workload then actually uses Caffe.

So of course, today it's no longer being as widely used, but it's very interesting to study the interface of the first generation deep learning framework.

Specifically, on those frameworks there's one core abstraction that is being used, which is called Layer.

So of course, in Caffe the layer is implementing in C++, but I'm just using Python pseudocode here to show the general idea.

So, when we are composing a neural network together, we will always talk about it like, "Hey, there's a linear layer, followed by a ReLU layer, and then it has Softmax." So, you can find that layer is kind of like the basic element you can use to compose them together.

And this compositionality is really helpful, so that for example I can, not only develop ReLU neural network, but we can replace all the ReLU activations by say sigmoid activation with a few lines of code.

And then that new neural network can be, you know, can be trained in the same fashion, right?

So, a layer interface defines two key functions.

The first function is what is called forward function.

In a forward function what it tries to do is try to take data from input, in this case it's called bottom, and try to propagate the data and the computation to output.

But if my layer is an exponential layer, on the left, what I try to do is it will do bottom exponential, and we are going to assign that to top here.

And in this case, both top and bottom have the memory already pre-allocated.

So in fact, they just run the computation and copy the data onto the top.

Of course, this is only running a forward computation.

In order to compute the updates, we will still need to do the gradient computations, and set the weighted gradient, use the weighted gradient to do the update.

So, that's where the backward function comes in.

And usually in Caffe, so the top contains the gradient from the output, and there's a propagate down field that uses to indicate, you know, whether or not we want to take, we want to propagate a gradient to the input because if say, on v1 you don't have to propagate gradient to the input at least, in this case, because you know, unless you are doing things like neural style where you are taking gradient with respect to input.

So, this propagate_down, it's just a specific flag that help us to do certain optimizations.

And bottom contains the array that allows it to propagate gradient onto.

Of course, in a backward function it's also going to refer to some of the activations and data from a forward propagation.

So, when in a backward function if I'm doing things exponential, ideally what you are going to do is if you are you are going to say my bottom

gradient is going to equal, in this case I do have a top value, which is exponential, times the top gradient in here.

That that is the gradient of exponential.

So, one thing you'll notice is that in this interface we are effectively doing backpropagation in place.

So, for this combination graph for example, there will be a multiply layer, an add layer, and exponential layer in here that composes together in this way.

And then in the neural network training, first of all, we're going to create all the layers, call forward in topological order so that we will be able to get all the intermediate activations, and then we are going to do reverse topological iterations and call backward function to propagate the gradient back.

After that, each of the layers will contain the gradient with respect to their internal weights.

And then we can call sometimes the update function, or solver function, to take the gradient and weights, and update the weight in place so that we can get effective weight updates.

And in the next iteration we're going to do that again, and again.

So, if you look at this interface, right?

You can find that it's slightly different from what we talk about in our automatic differentiation lecture.

Instead of taking the combination graph and extend another combination graph that performs the gradient calculations, this simply builds up the layer interface and propagates in place.

So, what are the advantages and disadvantages of this layer interface?

Or you know, another question that I want everybody here to think about is why did the author of Caffe and Cuda-Convnet come up with this layer interface?

Well, right now if you look at the design choices, maybe you know, it seems not as obvious to you because you are getting more used to the computational graph, automatic differentiation anyway.

But imagine that you have not learned about any of those concepts, and you just learned about the forward propagation, and back propagation in deep learning.

Very likely you are going to come up with some interface that looks like this one because it's quite natural, right?

You get layers, in each of the layers you can do two actions, you can do forward propagation and then you can do backward propagation to get a gradient.

I personally also developed my own libraries at the time, trying to work on some of the deep learning workflows, and I also come up with a similar interface that contains a forward and backward function.

So, it's kind of a quite natural choice, and if you look at the description of back propagations by Geoffrey Hinton and others, initially you'll find that it's also quite a natural choice to end up with in the beginning.

So, this style of interface kind of became popular, and became a standard of the first generation of deep learning framework.

So, after 2015 you'll find that there's a secondary interface, which is TensorFlow interface.

That starts to have the concept of computational graph instead of layers, right?

So, the difference of computational graph and layers is that in the computational graph we are simply describing the forward computations,

and then when we're taking gradient, we are going to extend the computational graph.

And so, TensorFlow is typical representor of this style of API, and specifically TensorFlow 1.0.

So, if you talk about TensorFlow now, it's already at 2.0.

So you know, a lot of things have changed, and then TensorFlow have already also adopted some of the imperative programming APIs that PyTorch uses.

But we are going to focus on the TensorFlow 1.0 API because it kind of represents a very interesting way of designing programming abstractions for machine learning.

I also want to mention that actually TensorFlow 1.0 is also not the first framework that come up with the content concept computation graph.

It's an early pioneer called Theano from the University of Montreal.

And you know, the concept of combination graph actually was pioneered by Theano.

So, you can see quite a few reflections of the API design between TensorFlow and Theano in the early days, okay?

So, let's take a look at how we are going to take a deep learning model and try to describe that using the computational graph API in TensorFlow 1.0.

And let's walk it through line by line.

So, to describe a combination graph, first all we are going to create a variable in here corresponding to v1 which says, you know, this is an in-place placeholder.

We do not yet know what the value is, but we want to kind of describe this computational graph.

In a second step, what we're going to do is we're going to say v_2 equals $tf.exp(v_1)$.

That's going to create this computational graph node.

Then I'm going to do v_3 equals $v_2 + 1$.

That's going to create a v_3 node.

Finally, v_4 equals $v_2 * v_3$ and that's going to create my v_4 node here.

Up until now, no computation have yet happened.

All this coding here simply construct the computation graph on the left hand side.

And you know, if you want to ask what is the value of v_1 ?

Or what is value of v_2 ?

The answer is we don't know.

We don't yet know, right?

Because only the API, only that who constructed this computational graph, and that's it.

So, in order to really go and execute we want to ask, what is the value of before the v_4 if v_1 is one?

What you need to do is need to create a session that helps you to send the execution command onto, and then you're going to see session run v_4 saying,

that hey, I'm interested in the value of v_4 and there's a `feed_dict` that that tells the system, okay, the value v_1 here should be one.

And then after run a session you will get the value of v_4 back in here, okay?

So, you can find in this API, it's a typical way of what we call a declarative program.

So, what does the declarative programming mean?

So, it means that we try to declare the computation, right?

In this case, it first declares the computation that we want in here, and then we have a second execution part that takes that declarations, and goes ahead and send the declarations onto the system.

So, let's pause a bit and think about, "Hmm, why does TensorFlow design API in this way?"

What are advantages of this kind of declarative API?

And also remember that, you know, this is an API design by Google, and Google is very well known for designing scalable systems.

So, there must be some considerations that comes into designing the neural network APIs in this way, right?

So, if we pause a bit, and think about, you know, what are the advantages of this declarative programming API, a few things you might notice.

The first thing is that in this code there's a very clear separation that you have a declaration part

that declares the complete entire computational graph, and you have a run part in here.

So, one of the advantages that when we go and execute the computations, we kind of know the entire computational graph already, right?

So, we can do certain kind of optimization.

So, imagine that in here, let's say I declare this computational graph.

Instead of saying, "Hey, I'm interested in getting a value of v4." What if I'm saying that I don't interested in v4 I'm only interested in v3.

What's going to happen is that the underlying system can go and analyze, okay, you have a declaration that contains all the computation graphs, but you are only interested in v3 but not v4.

So, how about let's go and only execute this part of the computation graph instead of all the components, right?

So, it gives you opportunity to look at a computational graph and skip some the computation that are unnecessary.

And it does happen, for example, if you have a forward computational graph and you take gradient with respect to that, imagine that you're only interested in gradient values but not interested in say, computing the loss function values.

Actually, you can skip the loss function computation in a lot of cases.

Another advantage of this is that effectively having a complete computational graph gives you a lot of opportunities for optimization.

Another thing that you might notice, and that's the creation of Session here.

And that that programming model effectively means that the place where you run this combination graph does not have to be on the same local machine that runs this Python code.

And you know, you can kind of imagine why, you know, why this design is being brought up by Google.

Because in Google it's really important for them to be able to have the scalable computations, and also being able to have, you know,

the execution part say on on different machine, or on on our powerful machine while we can do some of the developments on the local laptop.

So, that was one of considerations.

Imagine I have a local laptop that don't even come with a GPU, and I have a remote session that contains a GPU.

What you can do is you can go ahead and send the data to the remote, there's a remote executor that really go and run the computational graph, and then it gives you the computational of graph result back in value form.

So, these complete declarations and the separation of the computational graph from the current interactive host APIs kind of helps in that perspective, right?

It helps you to be able to organically send values to a different place where real execution happening, but you still do some of a lightweight, you know, debugging and other things on your local session.

One of the things we all know is that, you know, today TensorFlow, is still be widely used, but a lot of people prefer another kind of programming abstraction, which is PyTorch.

And let's first take a look at, you know, what PyTorch's programming abstraction is and let's come back and compare this declarative programming abstraction and PyTorch's programming abstraction.

So, in PyTorch and `autograd` as well, we're also adopting this imperative programming style.

You can find that in order run these computations, we are simply constructing the code like in here, right?

So, we'll first declare the `nd1`. `Tensor`, and then you compute exponential function, do a plus one, and then you multiply the things together.

On one hand, you know, the computation graph on the left hand side looks similar, and so, what's happening behind the scenes?

and what's the difference between this imperative automatic differentiation API versus the declarative computational graph API?

It's that besides computing the computational graph, when we do `v1` we concretely specify this is value one in here.

And then in the next step I not only create a computational graph node, but we are also computing exponential `v1`.

You know, exponential one is really hard to compute, but let's imagine this is zero and we will know that this is one here, right?

And we add it, it's going to be two, and at this point we will know that this is going to be two in here.

So, at each of the time steps not only we are constructing a computational graph, but we are also computing the data along the way.

And we have introduced needle implementation as well.

We know that, you know, in certain cases we are eagerly evaluating that, in other cases we are using this lazy Tensor approach where you don't necessarily have to eagerly evaluate that, but you know, you will be able to lazily evaluate the computations and get the data out.

So, the difference between this imperative automatic differentiation API or in other cases, it's called define and the run, right?

So, define by run.

So, as you define computations, we also go ahead and running the result.

It allow us to execute the computations as we construct the computational graph, and also allows us to easily mix the Python control flow and the construction at the same time.

We could write a code like this, but we can say that if `v4.numpy` is bigger than 0.5, I'm going to do `v5 equals v4 times two`, otherwise `v5 will equal v4`.

So, we can find that depending on the values of my intermediate computations, we could choose to dynamically construct my computational graph in one way or another.

So, people also commonly refer to this as a dynamic computational graph construction, as opposed to in a case of TensorFlow APIs, it's called static computational graph construction.

This imperative automatic differentiation API is also not first being brought up by PyTorch.

The earlier pioneer here is a framework called Chainer that first introduce this concept of define by run, and PyTorch really took that idea and you know, further developed it.

You know, and that becomes one of the most widely used deep learning framework today.

So, let's also pause a bit and try to compare and contrast.

What is the difference of this imperative automatic differentiation versus the declarative programming style that we had in TensorFlow?

You will find that there are a few things that pop up.

First of all, in terms of the optimization opportunities actually, you know, programming TensorFlow when possible gives us more opportunity for optimization.

For example, if I'm going to go ahead and program our things in this way, effectively, if I do eager mode computations, every time if we compute this, I will have to go ahead and really get this value out, right?

So, we will miss this opportunities like for example in the future, if we look at this entire computation graph, maybe we can fuse those computation together because you know, interactive environment is really hard to know whether or not this value is going to be referred by another component, right?

Also, if we do this and say, "Hey, I'm only interested in `v3`." Of course the `v4` computation already got executed, so you cannot do things like that code eliminations in here.

So, you know, in terms of optimization opportunities, the TensorFlow 1.0 actually is better in a sense.

That is why, you know, in a lot of cases people still prefer TensorFlow 1.0 in production because of the case.

And also, you know, if look at some of the machine learning inference frameworks or deployment frameworks, a lot of deployment frameworks still tries to take this computational graph declaration execution approach because that gives them more opportunities for things like memory reuse and other optimization as well.

However, you know, why do people still want to use the PyTorch API?

This is because, you know, it's much more friendly in certain cases, and it allows more organic mixing of Python, which is what we call host language, as well as underlying computation graph.

Like computation graph, we can view computation graph as another programming language effectively if you look at it.

Here the first component effectively is what we call a TensorFlow programming language that tries to describe the computational graph in here.

Then, you know, you can only declare and then run a computation.

So, imagine that you wanted to do things like if v_4 is bigger than 0.5 we are going to do one kind of computation, otherwise we're going to do another.

In TensorFlow you have to introduce a special kind of node like `tf.if` or `tf`.

There's also loops within the TensorFlow computation graph in order to do some kind of computations, which is a bit cumbersome.

Also in terms of debugging, in a lot of cases when we go ahead and build neural networks, right, it's not necessarily that you're going to get right all the time.

So, it could be very possible that you are going to take v_2 and we're going to ask you, "What's the value of v_2 ?"

So, people will come when I will do things like `print v2` in here, but at this time point we do not know what's the value of v_2 right?

Because you don't even know what's the value of V_1 in here.

So, you know, it's really hard to kind of debug the programing this way.

In PyTorch or in imperative AD APIs you can directly go ahead and insert the `print v2` here because, you know, as we execute to this time point, the value of v_2 is already known if we realized the underlying cache.

That gives the opportunity to be able to, you know, interactively debug.

The ability to mix in the Python code and the computational graph construction is also very helpful.

There are advance models like there's this model called stochastic depths, where in each of our iterations we're going to randomly flip

a coin and try to construct, you know, neural networks of different depths.

And that's really hard to be expressed in a static computation graph construction work because you know, effectively every time the optimization graph is changing, right?

Depending on the runtime solutions of the values and this happens in things like for example, natural language possessing in some cases where your computational graph, if you unroll things around a sequence length it can depend on the length of input, right?

So, the flexibility of this imperative automatic differentiation that allows you to organically construct these dynamic computational graphs.

It really gives the researchers much more ability, a bigger amount ability to be able to express the ideas easily in Python, right?

And that really leads to the adoption of the framework.

So, you can find that there's indeed some form of trade offs between the flexibility and ability to optimize.

You know, later there are techniques that try to get even this imperative programming model into a more, you know, efficient state.

And there are efforts like compilations, just in time compilation other efforts that that help us to move along the way.

And so, today if you look at the choices of programming abstractions you'll find that this imperative automatic differentiation is still being, you know, is being widely preferred in this case.

And it is really interesting combinations of, you know, the ease of use, the ability to be able to express the models that were interested in, and still being able to leverage underlying acceleration libraries to get the good amount of speed ups.

Okay?

So, until now we have talked about these programming abstractions.

We have studied three of them.

Caffe, which represents the first generation deep learning framework that allows you to do, you know, forward and backward propagation operations.

TensorFlow, that that gives you the ability to be able to construct a computational graph, and then, you know, execute them.

Also we can do automatic differentiations like extend and computation graphs. And PyTorch, which gives you the ability to be able to,

you know, imperatively compute intermediate computations and dynamically construct the computational graphs.

It will be useful, we have discussed the pros and cons, but it will be useful for all of us to take a moment

and pause a bit, and think about what are pros and cons of each of our program abstractions.

And imagine that, you know, you fly back to the time where, in say 2013, there's not yet a lot of choices for deep learning frameworks.

What kind of interface that you would design to resolve the problem of both modular construction of models, as well as expressing different kind of optimization algorithms and needs?

And it would be a very useful thought exercise that I would encourage everybody here to do.

Okay, so now we've talked about the high level programming abstractions.

Let's come back and think about, you know, imagine that now we have chosen the imperative automatic differentiation or dynamic computational graphs as our programming model.

And you know, you have already started to work on needle as the first part of a framework.

As first homework that gives you the ability to be able to construct a tensor through do arithmetic operations upon them, as well as composing those tensors then to automatic differentiation.

However, you know, we have not yet talked about how can we construct a neural network from all those tensors.

Of course we can do that, you know, using the lowest level API.

We can take the tensors, we'll be able to, you know, do each of layers manually, just like what you did for homework zero.

However, there are better approaches that we can take in here.

And one of the things I want to remind everybody is that, you know, coming back to machine learning, right?

When we talk about machine learning it's not a single cluster of saying that you kind of have to take it, and you have to go ahead and implement every part of it.

So, machine learning itself is very modular.

Specifically, every machine learning reaction at least come with three element right?

The hypothesis class allows you to be able to take your input and gives you a prediction.

There's the loss function that takes the hypothesis, the label, and try to measure how good your prediction is.

And finally, this optimization measure that you know, for given loss function hypothesis class, is going to try to find optimized parameters within that space of possible choices.

You can find that at least there are three elements that are really independent from each other, right?

As we started to develop machine learning libraries, these components are going to translate onto modular component in code that we will be using on composing things up.

And this is even true before the age of deep learning.

And you know, we can pause the bit and think about how can we translate those modular components onto code?

For example, because we have this loss function, so certainly that's the possibility of being able to form machine framework to supply loss functions.

And these have been very useful.

And in another framework that I developed before called XgBoost, right?

Hopefully some of you also use that framework.

There's the ability for us to customize the loss function so that, you know, when we first developed XgBoost

we only supported I think regression analysis, and there are people that are interested in survival analysis of time series and they don't have to go ahead and build an entire survival tree boosting package from scratch.

They just need to go ahead and customize the loss functions to be able to do that.

So, that's one of the advantages on thinking about those modular components.

So, you can go ahead and change part of your system, hopefully with minor effort, but gives you something that's much more powerful.

If you think about the context of deep learning, you'll find that the deep learning even more modular

in the sense that not only they can have these three components, hypothesis class, loss function, optimization method.

The hypothesis class itself is very modular.

So, let's take this as an example.

You know, this is what you would typically see in our neural network model, right?

So, when you go ahead and look at different neural network implementations, sometimes you will see visualization in this style or in a similar style.

So, this is a visualization diagram that describes, I would say, depending on how you call it, usually we can call it a three layer residual neural network.

And it contains three residual blocks.

So, the input goes into first residual block, then feeding into a second residual block, third one, finally it's going to feed a fully connected linear.

There's a SoftMax CrossEntropy that help us to do a multi label classification, a SoftMax, that help us to do a classification in a SoftMax CrossEntropy fashion.

If you look at these diagrams, you do not yet know what exactly is each of the component respond, because when we build a needle library

the earliest thing we learn is tensors and operations on tensors, but residual block is not a tensor, right?

So, we want to expand it a bit.

Each of the residual blocks can be composed by input that gets feeded onto a linear layer, a ReLU, another linear layer.

And the idea of residual block is that not only will we want to be able to, you know, do the nonlinear transformation in this pathway,

we can also go ahead and directly add the original input onto this sequence within the box.

And this is one of the most important breakthroughs, most important developments in the history of deep learning, right?

They help us to build a lot of the deep neural network models, specifically in computer vision.

It also has other names like highway networks and if you look at recurrent neural networks, it's also kind of a similar style where,

you know, LSTMs have like a memory cell that memorizes the past here.

So, that's a residual block.

And if you look at residual block you will find that till we have those rectangular cells, right?

That it does not consist of tensors and tensor operations.

So, we can go ahead and extend them again.

So, each one linear layer in this case is going to contain an input, weight that contains a linear layer.

In some cases, a linear layer also comes with a bias term and in this case, we're simply skipping the bias term to keep things simple.

So you have a linear layer that goes into a matmul model module?

And what we're going to do is we're going to to transpose your weight and do matrix multiplication with that input, and that that gives you the output in here.

And this linear layer also follows the conventional convention like PyTorch and other library uses.

Similarly, a ReLU layer will take input, you know, try to take the maximum of input and zero.

That gives you output, Okay?

So, we can go ahead and expand, right?

We can expand every residual module in this diagram, and then expand every linear ReLU in this diagram.

Finally, you're going to get it much more deeper, right?

On a computational graph that contains many, many more operations.

And you can find this recursive decomposition is really modular.

And when we are trying to build up your neural networks we're not thinking in terms of, "Hey, I want to build one matrix in one location or another." Or thinking about those module blocks like linear.

Now we are composing those module module blocks together using things like residual modules and so on.

And this also help us to think about how we can build our interface design, right?

That helps us to be able to build up those modules, and then compose those modules together.

And that's one of the important part to consider when designing interfaces to construct neural networks.

If you're interested in learning more about ResNet there are two papers here that you can go ahead and read.

And the ResNet paper is actually one of most cited paper actually in deep learning as well.

So, we have talked about this compositional idea, and one of the key data structure here is one called nn. Module.

In the next lecture we're also going to implement the nn. Module.

If I've used PyTorch before you might be familiar with this concept.

So, effectively, each little rectangular block here can be viewed as a module.

And this big rectangular block is also a module.

So, when we design modules, effectively it's a way for us to be able to compose things recursively together.

It follows a common principle for tensor in, tensor out.

So, the principal says that, you know, a module should take usually, you know, if it's a sequential part, it takes one tensor input and it outputs one tensor.

In certain cases it might take, you know, two tensors, a few tensors and output tensors as well.

The tensor in, tensor out principle really helps us to be able to compose things together, right?

So, if every module takes a tensor as input and outputs a tensor, you will be able to take the output tensor and extend it to our next module.

So, helps it to do computations.

There are other things we need to support as a part of the nn. Module.

Specifically when I look at linear, you can find that we have kind of trainable parameters or weights, right?

That are attached to this module.

And that means that we want to be able to get you know, in order to be able to update the weight in our,

you know, our training procedures, we will need to be able to inspect those modules and get those trainable weight out from those module collections.

Of course, we also need to find a way to initialize the parameters here as well.

So, that's kind of nn. Module.

If you think about loss functions, we can actually view loss functions as a special kind of module as well.

Usually that's a module called SoftMax CrossEntropy that gives the hypothesis prediction, the label, and it's going to output a scalar value.

And you know, tensor of rank 0 can be viewed as scalar value.

So, it's also output tensors that that can be used to run predictions in here.

And it follows what we call tensor in, scalar out API convention, right?

So, you will be able to compose up the modules for getting, you know, input prediction, and then you compose that prediction with the loss function and label.

It gives you a loss function.

With this principle, it gives us a bit more composability.

For example, if you want to do multiple objective functions, imagine that, you know, I want to have a neural network that's able to both give

you bounding boxes, as well as classes in that bounding box.

That's one instance of multi-objective prediction.

What you are likely going to have is that you are going to have kind of one objective function module

that try to, you know, predict bounding box closeness, and another objective that predicts the class,

that measures how good your class predictions are, right?

And because we follow the tensor in, scalar out principle, so we can apply those two modules, and we can just add last one and last two together,

maybe in a weighted sum fashion, right?

So, we can leverage this modular feature to help us to directly add things together, and then you just call the value dot backward to compute the gradient values.

Yeah.

There's also another thing that we will consider, it's that in certain cases when you are doing training, and when you're doing inference,

that might be a slight difference in terms of, you know, what kind of operation you want to perform.

For example, in the case of the loss function management, of course in training you do want to compute output loss functions, but in predictions maybe you don't have to.

So, usually this module sometimes come with a training or inference mode.

So that, you know, allows you to have a slightly different behavior between training and inference here.

Okay?

That's loss functions.

Besides loss function, we still need to be able to for a given neural network models.

We want to be able to take in a model, and then be able to perform weight updates on it.

So for given model, we want to be able to, and they are implicitly weight attached to each of our modules,

we want to be able to first take a list of weight from the model themselves, and then want to be able to perform updates on those.

And this is usually encapsulated in an abstraction called optimizer, that takes in a list of weights, and sometimes you also need to keep track of auxiliary states, like for example momentum in here.

So in in these cases, right, the optimizer will need to be able to take in the model's query, how many weights are there.

So, the weights will include W_1 , W_2 , W_3 , and W_4 .

And then it will perform the updates in some of the rules.

Either SGD or or SGD with momentum, and Adam updates in here.

Another thing that optimizer we need to do is we need to be able to create and keep track of our auxiliary states.

For example, the model only come with the weight, but in order to optimize those models, those weights will need to create momentum variables in here.

And those momentum variables are being checked by the optimizers in their internal states as well.

One of the consideration of optimizer is that we want to consider regularization factors.

In certain cases for who maybe want to want to think about what is the weight, what is the value of the, you know, we want to apply L2 regulations for example to make sure that the weight value...

Each of the weight values L2 norm is not too large.

So, there are two ways that we can incorporate these regularization terms onto a framework.

The first way is we can implement it as part of loss function.

Som you just say loss equals loss plus λ L2 norm of the weight.

So, that is certainly possible.

And in certain cases, you kind of have to do that because in more complicated gradient calculations, like in certain cases people might want to be able

to compute the L2 norm of the gradient value.

In that case, you kind of need to directly do automatic differentiations, that incorporate it as part of your weighted loss function.

Another way is you can simply incorporate as your update rule, your optimize update well if only care about weight L2 regularization

you can directly change your update rule to incorporate what we call weight decay term.

That allows us to do this.

So in fact, this is a weight decay term that corresponds one to the L2 norm regularization that we talk about.

This is sort of very typical in a lot of the optimizers.

Besides the weight updates and the optimizers, another thing that we want to think about is how we're going to initialize each of the parameters in our space, right?

So, the initialization do need to consider, especially when need to consider all the magnitude of initialization because you are building a very deep neural network.

If you're visualizing the weight to be too small, after a few layers of propagations, your values will get in close to zero.

If I initializing the, the value to be too big, then after a few layer of propagations my value, our value will explode, right?

Will become very big.

So, most of neural network libraries come with a set of common initialization routines that try to control the magnitude of the weight depending on the neural network architecture.

So, that you know overall that that tries to make sure that your overall magnitude of values do not change very much.

And for different kind of weights in your neural networks, like whether it's a bias term that you just add off that activation value,

sometimes you might have running sum of mean and variance that help us to re-normalize your neural networks, as well as the weight for like fully- connected layers and convolutions.

There are different ways for us to initialize them.

Usually bias terms can be initialized to zero for example.

For convolution weight and matrix modification weight, we usually use uniform distributions.

or Gaussian distributions where the order of magnitude depends on the input and output.

And usually initialization can be folded in the construction phase of nn. Module.

Besides the elements that we talk about so far, one of the important element that we still need to take care of is the data loading.

So, not only want to be able to build a model, we also want to have an effective data loader that feeds the model training in efficient fashion.

And there are different ways we can do data loading.

For example, in certain cases we want to be able to pre-process and augment data set by randomly shuffling the input, and transform the input, and you know, different kind of data augmentations can account for significant portion of prediction accuracy boost actually in deep learning models.

So, actually if you look at most of the machine learning training pipelines, the data augmentation is a huge part of it.

When people do things like, you know, affine transformations or images, rotating, jittering to make sure that we get a bigger input space, and the model is being robust to those kind of transformations.

One of the things that you want to notice that this data loading augmentation is also compositional in nature, right?

So, we could just similarly build a module that simply do random rotation, and then another module that do things like crop and resize in a sequential manner.

And we can compose them together to get a more powerful pipeline.

So, you don't have to design a pipeline that's doing both, right?

So, you just need to have a single step pipeline that's one or another, and compose together to get your end to end data pipeline.

Okay, so coming back to summarize, we'll find that we have talked about a lot of things in the second part.

Talked about nn. Module, it help us to be able to compose up the neural network libraries.

We also talk about how nn. Module can help us to build the loss functions in here, by passing parameters, and gradients to optimizer.

We'll be able to go ahead and try out different kind of optimization methods.

There are different forms of initialization we can do to make sure that the weight values are being initialized correctly.

Finally, there are data loader and pre processing steps that help us to load the data, and augment the feed it to the training pipeline.

And this really, this summary really highlights how modular the deep learning models are, right?

So, that is really one of the reason why deep learning kind of dominates a lot of the machine learning field, in a sense that the cost of trying out new things is really low, right?

So, imagine that I only care about, you know, doing detections, the developer and the scientists do not have to go and rebuild the entire ResNet or mobile net from scratch.

You can be able to directly use a backbone of ResNet and mobile net, put a detection loss function in the end, and then you will be able to experiment the end to end pipeline there.

And if I'm only interested in developing new optimization method like a variant of Adam for neural network training, we just need to go ahead and replace optimizer part.

So, this modular nature is really amazing that help us to be able to build different deep learning solutions together, and and also put them together, right?

So, everything kind of composed well with each other so that we'll be able to, you know, customize one part while working together with the rest to get an end to end solution.

And that that is kind of like a, one of a secret sauce, why, you know, deep learning and a lot of deep learning frameworks actually kind of come back and dominates a lot machine learning.

Of course, you know, there could be other modular components that we have not yet covered today, including things like how you can do learning rate schedule.

There are components that might help with things like hyperparameter tuning and so on.

I would certainly love us to, as an exercise, come back and think about what are the possible modular components other than the things we talked about today.

So in the end, I want to come back and revisit the programming abstraction and now we have talk about the modular components,

and how we can use different kind of modules, right to compose the deep learning acceleration.

Let's come back and talk, revisit the abstraction again, and really trying to think and appreciate, you know, how this kind of layer wise composition helps.

You look at the Caffe 1.0 API, You can find that a layer API in here, at least couples two things, It couples the gradient computation and the module composition.

So, in order for us to find build the ResNet in here, well it's kind of a bit complicated because I can go ahead and introduce a residual layer, but then we will need to be able to define a gradient computation for a residual layer, which is kind of non trivial, right?

One of the key observation that we make in need, what I mean a second generation framework, is that we can separate the gradient computations and module composition in two levels of API.

In a case of need, and PyTorch as well the computational graph construction and automatic differentiation or gradient computation is being handled by tensors and computational graphs, among tensors.

However, in order to modulate, compose things together while introducing this concept nn. Module, and a module can contain things like residual module, that effectively is more cross grained than the lower level, tensor level operation.

So, by separating these two concerns of how we want to be able to compute a gradient, and directly relying on the underlying tensor library to do that, we will be able to focus on modular compositions in a much more simpler fashion by simpler defining the forward computations of all the intermediate modules.

And then you use automatic differentiation to handle all the gradient computations.

And that actually really helped us to be able to develop more models easily, and you know, this might maybe be obvious now there you have seen

the designs, but it is really interesting to come and reflect, you know, how the design decouples things,

and how this approach helps us to be able to not only do automatic differentiation more effectively, but also composes the modules here more organically.

Okay, so that is the end of today's lecture.

Thanks everybody for coming.

The next lecture we're going to really get our hands dirty, and implement some of the high-level designs we talk about this lecture.

I'll see you in the next lecture then.